Documento de contexto · Leia antes de me ajudar

Projeto Conecta Escola: colocar o site no ar na AWS

Eu sou estudante e estou fazendo um projeto para a escola. Este documento explica o que eu tenho, onde quero chegar e como você, IA, deve me guiar, um passo de cada vez.

Primeira mensagem que vou te mandar junto com este PDF

"Oi! Leia este PDF inteiro. Ele explica meu projeto e como quero que você me ajude. Antes de começar, me faça as perguntas da seção 3 e depois vamos para o Passo 1."

1. Como eu quero que você me ajude

Eu **não tenho muita experiência com tecnologia**. Nunca usei terminal Linux e não conheço bem a AWS nem o GitHub. Por isso, siga estas regras:

1. **Um passo de cada vez.** Me dê **uma** instrução ou **um** comando por mensagem e espere eu responder antes de seguir.
2. **Diga exatamente onde clicar ou onde colar.** Ex.: "No console da AWS, clique em EC2 → Instâncias...", "Cole isto no terminal preto da EC2 e aperte Enter".
3. **Comandos sempre em bloco de código**, prontos para copiar. Nada de "substitua X" escondido no meio: se eu tiver que trocar algo, avise antes, em negrito.
4. **Depois de cada comando, me peça para colar o resultado** (o texto que apareceu) e confira se deu certo antes de seguir.
5. **Explique em uma frase simples** o que cada passo faz e por quê.
6. Se algo der errado, **primeiro me peça a mensagem de erro completa**. Não chute soluções.
7. **Use os scripts que já vêm no projeto** (seção 6). Não invente outra forma de instalar (nginx, pm2, Node direto na máquina, Kubernetes...), a não ser que os scripts realmente falhem e você me explique o motivo.
8. **Antes de qualquer comando que apague algo** (`rm`, `down -v`, encerrar instância, apagar repositório), me avise e peça confirmação.
9. Na tela da AWS os nomes podem estar em **português ou inglês**; me diga os dois quando puder.
10. Responda em **português**.

2. O que é o meu projeto

O **Conecta Escola** é um site onde as famílias da minha escola (**E.E VILA SÃO JOAQUIM II**, em Cotia/SP) encontram profissionais da própria comunidade (eletricista, professor particular, confeiteira...) e onde os profissionais divulgam seus serviços. No dia da apresentação, vou projetar um **QR code**: os pais escaneiam, criam uma conta, um aluno aprova o cadastro pelo painel de administrador, e eles já podem usar o site.

Comecei com um rascunho no Replit. Meu mentor, que trabalha com infraestrutura, me passou uma **versão nova e completa** do projeto num arquivo `conecta-escola.zip`, já pronta para rodar num servidor. **É essa versão que eu vou usar.** A versão antiga não serve mais.

Tecnologia (para você entender, eu não preciso mexer nisso)

Site (frontend)	React + Vite + TypeScript + Tailwind, na pasta `web/`
API (backend)	Node.js 22 + Express + TypeScript, na pasta `server/`
Banco de dados	SQLite (arquivo), guardado num **volume Docker** chamado `dados`
Como roda	**Um único container Docker** (site + API + banco), via `docker compose`
Porta	Dentro do container: 3000. Na EC2: **80** (definido por `APP_PORT=80` no `.env`)
Reinício	`restart: unless-stopped`: se a EC2 reiniciar, o site volta sozinho
Plano B	Serviço opcional `tunnel` (Cloudflare Tunnel) que gera uma URL https pública: `docker compose --profile tunnel up -d`

Arquivos principais (na raiz do projeto)

```
conecta-escola/
├── Dockerfile              ← receita do container
├── docker-compose.yml      ← como ligar (porta, volume, restart)
├── .env.example            ← modelo de configuração (o .env real NÃO vai pro GitHub)
├── .gitignore              ← impede .env, node_modules, banco de irem pro GitHub
├── README.md               ← documentação técnica
├── PUBLICAR-NA-AWS.md      ← guia de publicação (mesmos passos deste PDF)
├── scripts/
│   ├── instalar.sh         ← instala TUDO na EC2 com um comando
│   └── atualizar.sh        ← puxa mudanças do GitHub e reinicia o site
├── server/                 ← código da API
└── web/                    ← código do site
```

3. Situação atual (me pergunte o que estiver faltando)

Item	Situação
Conta na AWS	✅ Tenho e consigo entrar no console
Instância EC2	✅ Já criei uma. **Me pergunte:** qual sistema (Amazon Linux 2023 ou Ubuntu), qual tipo (ex.: t2.micro, t3.small) e em qual região (canto superior direito do console; deve ser **São Paulo, sa-east-1**)
IP público da EC2	❓ **Me peça para conferir** em EC2 → Instâncias → detalhes → "Endereço IPv4 público"
Portas liberadas (Security Group)	❓ Provavelmente não. É o Passo 2
CloudShell	Subi um zip **antigo** lá, mas **não vou mais usar**. A EC2 vai baixar o projeto direto do GitHub
Arquivo `conecta-escola.zip` novo	✅ Tenho no meu computador

Item	Situação
Conta no GitHub	❓ **Me pergunte** se tenho e qual é meu **nome de usuário**
Repositório no GitHub	❓ Pode ser que eu tenha um com a versão antiga. Se tiver, o mais simples é criar um **novo** chamado `conecta-escola` (ou outro nome, se esse já existir)

4. Onde eu quero chegar

- O código no meu **GitHub**, num repositório **público**.
- O site rodando na **EC2**, em **segundo plano**, num endereço tipo `http://18.231.xx.xx` que **qualquer pessoa** consegue abrir, inclusive pelo 4G.
- **Quando eu alterar um arquivo no GitHub**, a EC2 se atualiza sozinha. O script confere a cada 2 minutos; o build leva mais alguns.
- Eu saber a **senha do administrador** para aprovar os cadastros.

5. Plano de passos (me guie nesta ordem)

1 Colocar o projeto no GitHub (pelo navegador)

1. github.com → **+** → **New repository** → nome `conecta-escola` → marcar **Public** → **não** marcar "Add a README" → **Create repository**.
2. Clicar no link **"uploading an existing file"**.
3. No computador: extrair o zip, abrir a pasta onde aparecem `Dockerfile`, `server` e `web`, selecionar **tudo** (Ctrl+A) e **arrastar o conteúdo** (não a pasta) para a página → **Commit changes**.

Conferir: a página inicial do repositório mostra `Dockerfile`, `docker-compose.yml`, `scripts`, `server`, `web` direto na raiz. Se aparecer só uma pasta `conecta-escola` com tudo dentro, está errado: o script espera os arquivos na raiz.

Arquivos ocultos como `.gitignore`, `.dockerignore` e `.env.example` também precisam subir. Se não aparecerem no repositório, me ajude a adicioná-los (Add file → Upload files).

2 Liberar as portas da EC2 (Security Group)

EC2 → Instâncias → clicar na instância → aba **Segurança** (Security) → clicar no grupo `sg-…` → **Editar regras de entrada** (Edit inbound rules) → adicionar:

Tipo	Porta	Origem	Para quê
HTTP	80	`0.0.0.0/0` (Qualquer local-IPv4)	Qualquer pessoa abrir o site
SSH	22	Prefix list `com.amazonaws.sa-east-1.ec2-instance-connect` (digitar "instance-connect" no campo). Se não aparecer: `0.0.0.0/0`	Eu entrar pelo console (EC2 Instance Connect)

Se a região não for sa-east-1, o nome da prefix list muda (ex.: `com.amazonaws.us-east-1.ec2-instance-connect`).

3 Entrar na EC2 pelo navegador

EC2 → Instâncias → marcar a instância → botão **Conectar** (Connect) → aba **Conexão de instância do EC2** (EC2 Instance Connect) → manter o usuário sugerido (`ec2-user` no Amazon Linux, `ubuntu` no Ubuntu) → **Conectar**.

Conferir: abre um terminal preto com algo como `[ec2-user@ip-172-31-x-x ~]$`.

Para colar no terminal do navegador: Ctrl+Shift+V ou botão direito → Colar.

4 Baixar o projeto e instalar

No terminal da EC2, **um bloco por vez**:

```
sudo dnf install -y git || sudo apt-get install -y git
```

Instala o git. No Ubuntu a primeira parte dá erro ("dnf: command not found") e a segunda resolve. É normal.

```
git clone https://github.com/MEU-USUARIO/conecta-escola.git
```

Troque **MEU-USUARIO** pelo meu usuário do GitHub (e o nome do repositório, se for outro).

```
cd conecta-escola
bash scripts/instalar.sh
```

O `instalar.sh` faz sozinho:

1. Instala o Docker. No Amazon Linux: `dnf install docker` + plugins `compose` e `buildx` baixados do GitHub. No Ubuntu: script oficial get.docker.com.
2. Cria 2 GB de swap se a máquina tiver menos de 2 GB de RAM (evita travar no build em t2/t3.micro).
3. Cria o `.env` com `APP_PORT=80` e uma **senha de admin aleatória**, salva também em `~/senha-admin.txt`.
4. Roda `sudo docker compose up -d --build`: monta e liga o site em segundo plano.
5. Cria um **timer do systemd** (`conecta-atualizar.timer`) que roda `scripts/atualizar.sh` a cada 2 minutos.
6. Mostra o endereço público e a senha.

Conferir: a primeira vez leva de 5 a 10 minutos. No final aparece:

```
==> PRONTO!
  Site:      http://18.231.xx.xx
  QR code:   http://18.231.xx.xx/qr
  Admin:     Login: admin@conectaescola.local
             Senha: (12 caracteres aleatórios)
```

> **Me lembre**
>
> de anotar a senha do admin num lugar seguro e de não postar ela em lugar público. Para ver de novo: `cat ~/senha-admin.txt`

5 Testar

1. Abrir `http://IP` no computador e no celular (pelo 4G também). Usar **http://**, não https://.
2. Criar uma conta de teste pelo celular.
3. Entrar como admin → **Administração** → **Cadastros** → **Aprovar**.
4. Abrir `http://IP/qr`: é a tela que vou projetar.

6 Testar a atualização automática

1. No GitHub, abrir `web/src/pages/Home.tsx` → lápis ✏️ → mudar um texto (ex.: "Laços mais fortes.") → **Commit changes**.
2. Esperar de 3 a 7 minutos e recarregar o site (F5).
3. Para acompanhar ao vivo: `sudo journalctl -u conecta-atualizar -f` (sair: Ctrl+C).
4. Para forçar na hora: `cd ~/conecta-escola && bash scripts/atualizar.sh`

O `atualizar.sh` faz `git fetch`; se houver commit novo, faz `git reset --hard` para a versão do GitHub (o `.env` e o banco são preservados) e `docker compose up -d --build`. Se o build falhar, a versão anterior continua no ar.

7 Antes da apresentação

- **IP fixo:** se a instância for parada e ligada de novo, o IP público muda e o QR antigo para de funcionar. Me ajude a criar e associar um **IP elástico** (Elastic IP) para o endereço não mudar.
- Opcional: um link **https** com `sudo docker compose --profile tunnel up -d` e depois `sudo docker compose logs tunnel | grep trycloudflare` (o link muda a cada reinício).
- Ensaio completo com colegas: cadastro → aprovação → busca → pedido de orçamento.

6. Comandos do projeto (rodar dentro de `~/conecta-escola` na EC2)

Comando	O que faz
`bash scripts/instalar.sh`	Instala e liga tudo (rodar **sem sudo**; pode repetir com segurança)
`bash scripts/atualizar.sh`	Puxa do GitHub e reinicia se mudou (`--forcar` reconstrói mesmo sem mudança)
`sudo docker compose ps`	Mostra se o site está rodando (Up / healthy)
`sudo docker compose logs app --tail 50`	Últimas mensagens do site
`sudo docker compose restart app`	Reinicia o site
`sudo journalctl -u conecta-atualizar -n 50`	Histórico das atualizações automáticas
`cat ~/senha-admin.txt`	Mostra a senha do admin
`sudo docker compose exec app node dist/reset-admin.js NovaSenha123`	Troca a senha do admin (mín. 8 caracteres)

Comando	O que faz
`sudo docker compose exec app node dist/backup.js`	Faz backup do banco (fica em `/data/backups` no volume)
`curl -s localhost/api/health`	Teste rápido: deve responder `{"ok":true}`
`sudo docker compose down -v`	⚠️ **Desliga e APAGA todos os dados.** Só no fim do projeto, com confirmação

7. Problemas conhecidos e como resolver

Sintoma	Causa provável → solução
EC2 Instance Connect não conecta	Falta a regra SSH 22 (Passo 2), a instância não tem IPv4 público, não está "Executando" (Running), ou o usuário está errado (`ec2-user` × `ubuntu`).
`git clone` pede usuário/senha ou dá "not found"	Repositório privado ou URL errada. Deixar **Public** e conferir usuário e nome na URL.
`scripts/instalar.sh: No such file or directory`	Pasta errada. Rodar `ls`. Se existir outra pasta `conecta-escola` dentro, entrar nela, ou corrigir o repositório (arquivos devem ficar na raiz).
"Rode SEM sudo"	Rodar `bash scripts/instalar.sh`, sem `sudo` na frente.
Build trava, "Killed", exit code 137 ou "heap out of memory"	Falta de memória. Conferir o swap com `free -h` e rodar o instalador de novo.
`permission denied … docker.sock`	Usar `sudo` antes de `docker`, ou sair e entrar de novo na EC2.
"compose build requires buildx 0.17 or later"	Plugin buildx não instalou (Amazon Linux). Pedir a saída de `docker buildx version` e reinstalar o plugin seguindo o trecho do `instalar.sh`.
Site não abre pelo IP	1) Regra HTTP 80 no Security Group? 2) Usou **http://**? 3) `sudo docker compose ps` mostra Up? 4) `curl -s localhost/api/health` responde?
QR code aponta para "localhost"	Abrir a página `/qr` pelo IP público, não por localhost.
Alterei no GitHub e não mudou	Esperar ~7 min. Ver `sudo journalctl -u conecta-atualizar -n 50`. Se houver erro de build, corrigir o código no GitHub (a versão antiga continua no ar).
IP mudou	A instância foi parada e ligada. Usar o IP novo ou associar um IP elástico.
Pai esqueceu a senha	No site: Entrar → "Esqueci minha senha". Admin → aba **Senhas** → Gerar link → enviar pelo WhatsApp.

8. Regras de segurança (não quebre estas)

- **Nunca** me peça para colocar senha, `.env` ou `senha-admin.txt` no GitHub. O repositório é público.
- Não abrir outras portas além da 80 (público) e da 22 (Instance Connect).

- Os cadastros são de **pais reais** (nome, e-mail, telefone). A página de privacidade do site promete apagar os dados no fim do projeto.
- **Depois da apresentação**, me lembre de: encerrar a instância (EC2 → Estado da instância → Encerrar), o que também apaga os dados; liberar o IP elástico, se tiver criado, porque ele cobra parado; e, se quiser, arquivar o repositório.
- A EC2 cobra enquanto estiver ligada. Me avise sobre custos se eu escolher algo diferente de t2/t3.micro ou t3.small.

> **Resumo para a IA**
>
> Guie-me **um passo por vez**, na ordem da seção 5, usando os scripts do projeto. Antes do Passo 1, me pergunte: (1) meu usuário do GitHub; (2) sistema, tipo e região da minha EC2; (3) se a EC2 tem IPv4 público. Depois de cada comando, peça o resultado e confira antes de seguir. Se der erro, peça a mensagem completa e use a seção 7.

Se travarmos em algo que nenhum dos dois consiga resolver, vou mandar o erro para o meu mentor.